Algorithm

간호사 근무표 생성을 위한 알고리즘.

Mike야 | 2025. 2. 17. 15:28

해당 알고리즘이 적용된 무료 간호사 근무 프로그램을 이용하여 근무표 생성의 고통을 덜어보세요! (모바일도 가능하나 pc 접속을 권장)
dutymate.net

간호사 근무표란?

- 간호사 근무표는 한 달 동안 각 간호사의 근무 일정을 한눈에 볼 수 있게 정리한 표입니다. 이 표는 각 날짜별로 누가, 어떤 유형의 근무(주간, 야간, 저녁 등)를 수행하는지 일목요연하게 보여줍니다.

이름	이전 근무				1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
김민성	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
간호사7	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
간호사8	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
간호사9	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
간호사10	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
간호사5	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
간호사6	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
간호사7	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
간호사8	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
간호사9	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-

1. 기술 도입 배경

간호사 근무표생성의 기본 원리에 대해 설명드리겠습니다. 위의 표는 간호사 근무 표 생성 전 단계입니다. 여기서 근무표를 완성하기 위해서는 약 300여 칸에 4가지 근무유형(Day,Evening,Night,Off)를 채워넣는 과정을 진행하게 됩니다. 저는 다양한 조건들을 만족시키며, 해당 규칙표를 자동으로 생성하는 프로그램을 작성하려 합니다. 근무표 생성에는 여러가지 조건이 붙습니다. 300여 칸에 4가지 근무유형을

채워 넣는 과정은 단순해 보이지만, 실제로는 2^300개 이상의 경우의 수가 존재하는 복잡한 최적화 문제입니다.

1. 제약조건

- 반드시 지켜야 하는 규칙들:
  - 각 근무조별 필요 인원수를 충족해야 함
  - 연속 근무 일수 제한
  - 필수 휴식시간 보장
  - 특정 패턴 금지(ND, NE 등등)
- 가능한 한 지키면 좋은 규칙들:
  - 개인 선호도 반영 (특정 요일/시간대 선호)
  - 공정한 근무 배분
  - 주말 근무 횟수 균등 배분
  - 숙련도에 따른 조별 인원 구성
  - 연속된 야간 근무 최소화

  첫 번째 시도는 그리디 알고리즘이었습니다. 매 순간 근무표에 점수를 부여하고, 해당 점수를 점진적으로 높여가는 방식으로 진행했습니다. 예를 들어, 최대 근무 일수가 5일이면 근무 일수가 높아질수록 패널티를 누적하고, 5일이 쌓이면 6일 차에는 매우 높은 패널티를 부여하는 방식으로 특정 패턴들을 제한하고자 했습니다. 이러한 그리디 방식은 2~3개의 제약 조건에서는 잘 작동했으나, 제약조건이 다양해지고 복잡한 규칙과 요청들을 수용하는 과정에서 점차 오류가 증가했습니다.

그리디 알고리즘의 주요 문제점:

1. 지역 최적해 문제: 매 순간의 최적 선택이 전체적인 최적해를 보장하지 못함
2. 제약조건 충돌: 다수의 제약조건이 충돌할 때 해결 방안 부재
3. 유연성 부족: 새로운 규칙이나 예외사항 반영이 어려움여러 알고리즘을 조사하고 시도한 끝에 **시뮬레이티드 어닐링** 알고리즘을 선택하여 적용하였고, 해당 모델이 효과적으로 작동함을 확인하였습니다.
4. 그 결과, 우리 서비스가 목표로 하는 병동별 맞춤 규칙 반영, 근무자들의 개별 근무 요청, 근무자들 간의 공정한 근무 배분이 사실상 불가능해졌습니다. 그리디 알고리즘은 항상 지역 최적해에 고립되어, 다양한 최적해를 찾지 못하는 경향을 보였습니다. 이에 알고리즘의 수정이 필요하다 판단하였고, 그리디의 한계를 극복할 수 있는 알고리즘을 조사하였습니다.

2. 시뮬레이티드 어닐링(Simulated Annealing)

시뮬레이티드 어닐링 알고리즘은 금속 재료의 어닐링(Annealing) 과정에서 영감을 받은 최적화 알고리즘입니다.

어닐링이란 금속을 고온으로 가열한 뒤 천천히 냉각시켜 결정구조를 안정화시키는 과정입니다. 이 과정에서 금속 원자들은 처음에는 높은 에너지로 자유롭게 움직이다가, 온도가 낮아지면서 점차 안정적인 상태로 정렬됩니다.

알고리즘의 작동 원리는 다음과 같습니다:

1. 초기 단계 (높은 온도)

- 무작위로 해답을 크게 변경할 수 있음
- 나쁜 해답도 높은 확률로 수용
- 다양한 영역 탐색 가능

2. 중간 단계 (중간 온도)

- 변경 폭이 적당히 제한됨
- 나쁜 해답도 일정 확률로 수용
- 지역 최적해 탈출 가능

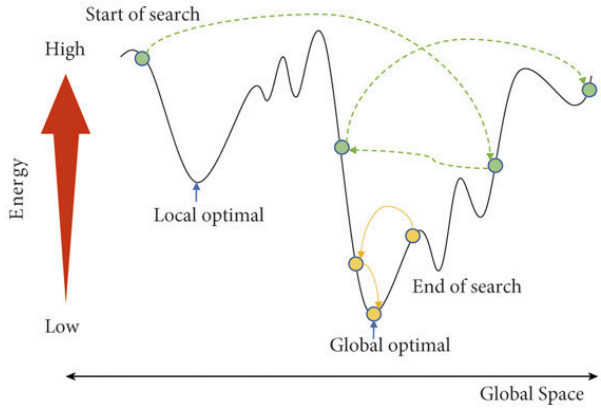
3. 마지막 단계 (낮은 온도)

- 아주 작은 변경만 허용
- 나쁜 해답은 거의 수용하지 않음
- 최적해 근처로 수렴

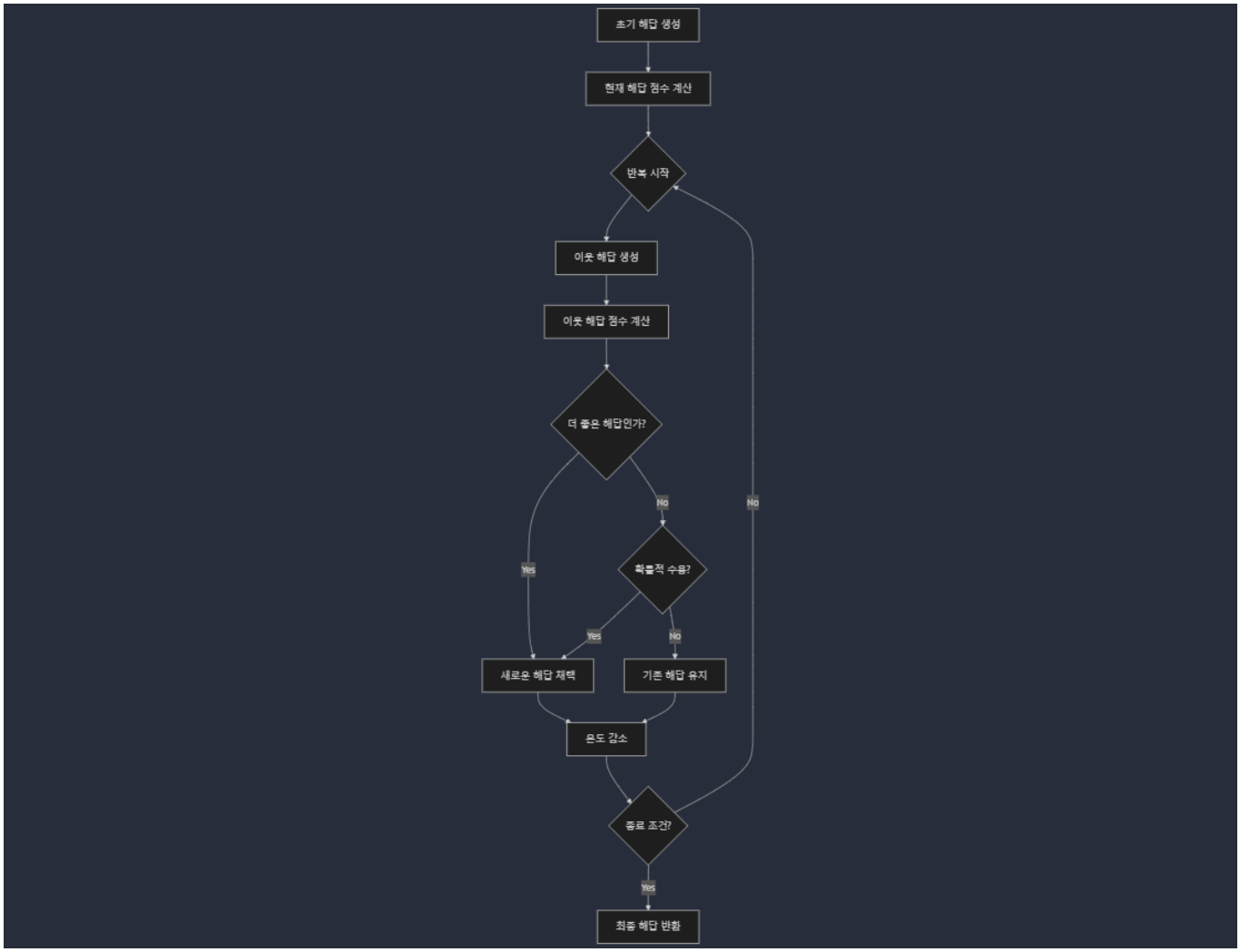
매 순간 알고리즘은:

1. 현재 해답을 약간 수정해 새로운 해답 생성
2. 새 해답이 더 좋으면 항상 채택
3. 새 해답이 더 나쁘면 확률적으로 채택 (온도가 낮을수록 채택 확률 감소)

이러한 특성으로 인해 단순한 그리디 알고리즘보다 더 넓은 해답 공간을 탐색할 수 있으며, 지역 최적해에 빠지는 것을 피할 수 있습니다.



https://www.researchgate.net/figure/Principle-of-the-simulated-annealing-algorithm_fig5_360434054



1. 초기 단계 설정초기 온도, 냉각률, 최대 반복 횟수를 특정 의도에 맞게 설정합니다.
2. `private static final double INITIAL_TEMPERATURE private static final double COOLING_RATE private static final int MAX_ITERATIONS`
3. 초기 해답 생성기존에 가지고 있던 근무표로 초기 해를 설정하고, 해당 근무표를 다양한 제약조건에 맞추어 점수화합니다. 이 점수를 기반으로 최적 해답을 찾아 지속적으로 갱신합니다.
4. `Solution currentSolution = createInitialSolution(wardSchedule, rule, wardMembers, yearMonth); Solution bestSolution = currentSolution.copy(); double currentScore =`

```
evaluateSolution(currentSolution, rule, prevMonthSchedules, shiftRequests); double bestScore = currentScore;
```

5. 반복적인 최적화 과정
6. `for (int iteration = 0; iteration < MAX_ITERATIONS; iteration++) { Solution neighborSolution = generateNeighborSolution(currentSolution); double neighborScore = evaluateSolution(neighborSolution, rule, prevMonthSchedules, shiftRequests); if (acceptSolution(currentScore, neighborScore, temperature)) { currentSolution = neighborSolution; currentScore = neighborScore; // 최적 해답 업데이트 } temperature *= COOLING_RATE; if (noImprovementCount > MAX_NO_IMPROVEMENT) { temperature = INITIAL_TEMPERATURE; noImprovementCount = 0; } }`

- 이웃 해답 생성: 5가지 금지 케이스 중 하나를 만족하도록 랜덤하게 생성하여 특정 지역해에 빠지지 않도록 유도합니다.
- 이웃 해답과의 비교: 생성된 이웃 해답의 점수를 계산하여 둘 중 하나의 해를 선택합니다.
- 온도 감소: 이웃 해를 수용한 경우 온도를 낮추어 특정 지역해에 수렴하도록 유도합니다.
- 개선이 없는 경우: 온도를 다시 초기 온도로 되돌리는 메커니즘을 추가하여 지역 최적해에서 탈출할 수 있는 기회를 제공합니다.

4. 해답 평가 시스템다양한 제약 조건에 가중치를 부여합니다. 병동별로 특정 조건에 대한 가중치를 다르게 설정할 수 있어 유연한 제약 조건 관리가 가능합니다.
5. `private double evaluateSolution(Solution solution, Rule rule, Map<Long, String> prevMonthSchedules, List<ShiftRequest> requests) { score += evaluateShiftRequirements(solution) * (병동별 제약 점수); score += evaluateConsecutiveShifts(solution, rule) * (병동별 제약 점수); score += evaluateWorkloadBalance(solution) * (병동별 제약 점수); // ... }`
6. 온도에 따른 수용 확률
7. `private boolean acceptSolution(double currentScore, double neighborScore, double temperature) { if (neighborScore < currentScore) { return true; } double probability = Math.exp(-delta / temperature); return random.nextDouble() < probability; }`

그 결과 다양한 조건과 요청을 수용한 최적의 근무표를 5초 이내로 자동생성 하는 것에 성공했습니다. 해당 병동에 적용된 자동 생성 규칙은 다음과 같습니다.

1. 근무 유형별 최소 인원 충족 (주간 : Day-3 Evening-2 Night-2, 주말 : Day-2 Evening-2 Night-2)
2. 최대 근무 일 수 5일 이하(병동별 커스텀 가능)
3. 특정 근무 요청 반영 (간호사 별 인당 2개의 요청까지 수용 가능)
4. 간호사별 균등한 업무 유형 배분
5. 특정 패턴 금지 (N ->D, N->E, NOD)

6. 나이트 단독 근무 or 나이트 연속 4일 이상 근무 금지(생활 리듬 유지를 위한 노력)

7. off를 몰아서 주는것에 작은 가중치 점수를 부여

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	D	E	N	O
N	N	O	O	E	O	D	N	N	O	E	E	N	N	N	O	E	O	E	O	D	D	O	O	D	D	D	D	E	O	D	8	6	7	10
D	O	E	O	D	D	D	O	N	N	O	O	D	D	D	O	E	O	E	E	O	E	O	D	D	E	N	N	O	N	N	9	6	6	10
O	D	E	E	O	O	D	D	O	E	E	O	D	D	E	O	D	D	D	D	E	O	N	N	N	O	E	N	N	O	O	9	7	5	10
O	E	O	D	D	D	O	E	E	E	O	D	D	N	N	N	O	O	D	D	E	E	O	N	N	N	O	O	D	D	O	9	6	6	10
O	D	D	D	D	O	E	O	O	D	N	N	N	O	O	E	O	E	N	N	N	O	E	O	D	O	D	E	O	E	E	7	7	6	11
E	O	D	N	N	N	O	E	O	N	N	N	O	O	D	D	O	E	O	D	D	O	D	D	E	E	O	E	E	O	D	8	7	6	10
O	E	O	E	N	N	N	O	E	O	O	D	O	D	O	O	D	D	D	E	O	D	D	D	E	O	E	O	N	N	N	8	6	6	11
E	O	D	D	O	D	O	D	D	D	D	E	O	E	O	N	N	N	O	O	D	N	N	O	O	N	N	O	O	E	E	8	5	7	11
D	O	N	N	O	E	E	O	D	D	D	D	E	O	E	E	N	N	O	O	N	N	O	E	O	D	D	D	D	O	O	9	6	6	10
N	N	N	O	E	E	N	N	O	O	D	O	E	E	O	D	D	D	N	N	O	O	E	E	O	D	O	D	O	D	D	8	6	7	10
2	2	3	3	3	3	3	2	2	3	3	3	3	3	2	2	3	3	3	3	3	2	2	3	3	3	3	3	2	2	3				
2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2				
2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2				
4	4	3	3	3	3	3	4	4	3	3	3	3	3	4	4	3	3	3	3	3	4	4	3	3	3	3	3	4	4	3				
10	10	10	10	10	10	10	10	10	10	10	10	10	10	10	10	10	10	10	10	10	10	10	10	10	10	10	10	10	10	10				

100%
완성도

3 구독하기

'Algorithm' 카테고리의 다른 글

Comparable vs Comparator (0) 2025.03.10

java 이슈 모아보기 (3) 2025.01.14

'Algorithm' Related Articles

Comparable vs Co...

java 이슈 모아보기